

Gestiona Gateway API Documentation

Versão 1.5.0

Index

Models

- UploadDocumentRequest
- CreateProcessRequest
- GatewayResponse
- UploadDocumentResult
- UploadDocumentError
- ThirdResult
- ThirdError
- ProcessResult
- ProcessError
- ProcessThirdsResult
- ProcessThirdsError
- ProcessDocumentItem
- ProcessDocumentsError
- Download Success Output

Shared

- Request headers

Endpoints

- 1. POST /processes
- 2. POST /processes/documents?process_number=<numero>
- 3. POST /processes/{process_id}/documents
- 4. POST /processes/documents/{folder_id}?process_number=<numero>
- 5. POST /processes/{process_id}/documents/{folder_id}
- 6. GET /documents/{document_id}
- 7. GET /thirds?nif=<nif>
- 8. GET /thirds/{third_id}
- 9. GET /processes?process_number=<numero>
- 10. GET /processes/thirds?process_number=<numero>
- 11. GET /processes/{process_id}/thirds
- 12. GET /processes/{process_id}/documents
- 13. GET /processes/{process_id}/documents/{document_id}

Models

UploadDocumentRequest

Used as the request body for both upload endpoints.

```
{
  "operationId": "string | null",
  "id": "string | null",
  "name": "string | null",
  "fileName": "string | null",
  "documentSourceType": "string | null",
  "url": "string | null",
  "content": "string | null"
}
```

Field notes

- `documentSourceType` Expected values in the current implementation are `DIGITAL`, `EXTERNAL_URL`, and `FOLDER`.
- `fileName` Used for `DIGITAL` uploads when the file is read from local storage.
- `content` Base64-encoded file content for `DIGITAL` uploads.
- `url` External URL used when `documentSourceType` is `EXTERNAL_URL`.

CreateProcessRequest

Used as the request body for POST `/processes`.

```
{
  "activityId": "string",
  "procedureId": "string",
  "userId": "string",
  "groupId": "string",
  "freeSubject": "string"
}
```

Field notes

- `activityId` Sent upstream as the Gestiona catalog procedure id in `catalog-2015/procedures/{activityId}`.
- `procedureId` Sent upstream as the Gestiona external procedure id in `external-procedures/{procedureId}`.
- `userId` Used to build the file-opening initial assignation link: `{GestionaApiBaseUrl}/users/{userId}`.
- `groupId` Used to build the file-opening management unit group link: `{GestionaApiBaseUrl}/groups/{groupId}`.
- `freeSubject` Sent upstream as `free_title`.

GatewayResponse

Used as the response envelope for gateway success and error responses.

```
{
  "operationId": "string | null",
  "success": true,
  "result": {}
}
```

UploadDocumentResult

Used inside `GatewayResponse.result` on upload success.

```
{
  "id": "string",
  "processId": "string",
  "creation_date": "string",
  "modification_date": "string"
}
```

Field notes

- `id` The API returns the created Gestiona entity id. If the upstream create response does not include `id`, the service resolves it from the last path segment of the `self` link in the upstream `Links` collection.

UploadDocumentError

Used inside `GatewayResponse.result` on upload errors and download errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for upload

- Configuration
- Validation
- NotFound
- Upstream

Possible kind values for download

- Configuration
- Validation
- NotFound
- Upstream

ThirdResult

Used inside `GatewayResponse.result` on third lookup success.

```
{
  "full_name": "string | null",
  "first_name": "string | null",
  "second_surname": "string | null",
  "nif_country": "string | null",
  "id": "string | null",
  "nif": "string | null",
  "type": "string | null",
  "email": "string | null",
  "mobile": "string | null",
  "nif_type": "string | null",
  "address": "string | null",
  "number": "string | null",
  "zip_code": "string | null",
  "province": "string | null",
  "country": "string | null",
  "type_of_road": "string | null",
  "zone": "string | null",
  "parish_code": "string | null"
}
```

Field notes

- Address fields are obtained from Gestiona GET `/thirds/{third_id}/default-address` after the base third is retrieved.

ThirdError

Used inside `GatewayResponse.result` on third lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for third lookup

- Configuration
- Validation
- NotFound
- Upstream

ProcessResult

Used inside `GatewayResponse.result` on process lookup success.

```
{
  "id": "string",
  "processNumber": "string"
}
```

Field notes

- `id` The resolved Gestiona file id.
- `processNumber` The process number used to resolve the file id.

ProcessError

Used inside `GatewayResponse.result` on process lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for process lookup

- Configuration
- Validation
- NotFound
- Upstream

ProcessThirdsResult

Used inside `GatewayResponse.result` on process thirds lookup success.

```
{
  "processId": "string",
  "thirds": "third-id-1;third-id-2"
}
```

Field notes

- `thirds` Semicolon-separated third ids extracted from each upstream `rel: third` link.

ProcessThirdsError

Used inside `GatewayResponse.result` on process thirds lookup errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values for process thirds lookup

- Configuration
- Validation
- NotFound
- Upstream

ProcessDocumentItem

Used for each item in `GatewayResponse.result` returned by the process document-listing endpoints.

```
{
  "type": "DOC",
  "name": "POC_SIGMA_Gestiona",
  "id": "f5e0364b-9951-449d-8d38-34a6cbfec4d3"
}
```

Field notes

- type The upstream item type, such as DOC or FOLDER.
- name The upstream rel value.
- id The last path segment of the upstream href.

ProcessDocumentsError

Used inside `GatewayResponse.result` when a process document-listing request fails.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible kind values

- Configuration
- Validation
- NotFound
- Upstream

Download Success Output

The download endpoint does not return JSON on success. It returns the raw document bytes in the response body.

Relevant response characteristics

- Body: binary file content
- Content-Type: document MIME type returned by Gestiona, or application/octet-stream as fallback
- Content-Disposition: attachment, with automatic download filename

The underlying DLL model used by the service layer is

```
{
  "documentId": "string",
  "fileName": "string | null",
  "contentType": "string | null",
  "storageSize": 0,
  "storageExtension": "string | null",
  "storageMimeType": "string | null",
  "storageMd5": "string | null",
  "storageSha1": "string | null",
  "storageSha512": "string | null",
  "content": "byte[]"
}
```

Request headers

All endpoints can optionally receive:

- `X-User -Access-Token` When present and not blank, the gateway uses this value as the upstream `Gestiona X-Gestiona-Access-Token`.

If `X-User -Access-Token` is absent or blank, the gateway uses the configured token from `Gestiona:AccessToken`.

Endpoints

1. POST /processes

Creates a new Gestiona process by creating the upstream file from the catalog/external procedure pair and then opening that file through the file-open link returned by Gestiona.

Query parameters

- `operationId` optional

Request body model

- `CreateProcessRequest`

Request body example

```
{
  "activityId": "82722c9b-cecc-4299-8a7b-ce5abeb8170b",
  "procedureId": "external-procedure-id",
  "userId": "3f18aa72-4091-4a50-8210-6c05ca234647",
  "groupId": "2bb7ddb2-a870-470d-ae87-3d4b2d3dd4af",
  "freeSubject": "Mais um processo criado a partir da API"
}
```

Upstream calls

1. POST /catalog-2015/procedures/{activityId}/external-procedures/{procedureId}/create-file
2. POST {file-open href returned by create-file}
 - `Content-Type`: `application/vnd.gestiona.file-opening+json; version=1`
 - `entry_date` is copied from the create-file response.
 - `free_title` is copied from `freeSubject`.
 - `initial_assignment[0].href` is `{GestionaApiBaseUrl}/users/{userId}`.
 - `links[0].href` is `{GestionaApiBaseUrl}/groups/{groupId}` with `rel` equal to `management-unit-group`.

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `ProcessResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "Id": "a7a43429-a82c-4245-9f50-f1e853905a99",
    "processNumber": "16/2026"
  }
}
```

Error response

- HTTP 400, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `ProcessError`

Notes

- The first upstream response must include `entry_date` and a link with `rel` equal to `file-open`.
- The second upstream response must include both `id` and `code`.
- `ProcessResult.Id` is mapped from the second upstream response `id`.

- `ProcessResult.processNumber` is mapped from the second upstream response code.

2. **POST** /processes/documents?process_number=<numero>

Creates a document by resolving the target Gestion process_id (file id in gestion) from the query parameter `process_number`.

Query parameters

- `process_number` required

Request body model

- `UploadDocumentRequest`

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. GET /files with filter `exact_code = process_number`
2. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{resolved_process_id}/documents-and-folders
3. For FOLDER requests:
 - POST /files/{resolved_process_id}/documents-and-folders
4. For EXTERNAL_URL requests:
 - POST /files/{resolved_process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `UploadDocumentResult`

Success example

```
{
  "operationId": "op-123",
  "success": true,
  "result": {
    "id": "document-id",
    "processId": "file-id",
    "creation_date": "2026-05-08 10:00:00",
    "modification_date": "2026-05-08 10:00:00"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 400,
    "name": "Bad Request",
    "kind": "Validation",
    "message": "process_number query parameter is required."
  }
}
```

3. POST /processes/{process_id}/documents

Creates a document directly in the Gestion file identified by `process_id`.

Route parameters

- `process_id` required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{process_id}/documents-and-folders
2. For FOLDER requests:
 - POST /files/{process_id}/documents-and-folders
3. For EXTERNAL_URL requests:
 - POST /files/{process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: UploadDocumentResult

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Notes

- For DIGITAL uploads, either fileName or content must be provided.
- If both fileName and content are provided, the current implementation uses content.
- On successful create operations, result.id may come either from the upstream id field or, when that field is missing, from the last segment of the upstream self link.

4. POST /processes/documents/{folder_id}?process_number=<numero>

Creates a document inside the Gestion folder identified by folder_id, after resolving the target Gestion file from the query parameter process_number.

Route parameters

- folder_id required

Query parameters

- process_number required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. GET /files with filter `exact_code = process_number`
2. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{resolved_process_id}/documents-and-folders/{folder_id}
3. For FOLDER requests:
 - POST /files/{resolved_process_id}/documents-and-folders/{folder_id}
4. For EXTERNAL_URL requests:
 - POST /files/{resolved_process_id}/documents-and-folders/{folder_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: UploadDocumentResult

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Notes

- This route uses the same document creation flows as the process-number route, but targets the upstream Gestiona endpoint with the folder id in the last path segment.
- For DIGITAL uploads, either `fileName` or `content` must be provided.
- If both `fileName` and `content` are provided, the current implementation uses `content`.
- On successful create operations, `result.id` may come either from the upstream `id` field or, when that field is missing, from the last segment of the upstream `self` link.

5. POST /processes/{process_id}/documents/{folder_id}

Creates a document directly inside the Gestiona folder identified by `folder_id`, under the file identified by `process_id`.

Route parameters

- `process_id` required
- `folder_id` required

Request body model

- UploadDocumentRequest

Request body examples

DIGITAL

```
{
  "operationId": "op-123",
  "name": "Contrato",
  "fileName": "contrato.pdf",
  "documentSourceType": "DIGITAL",
  "content": "JVBERi0xLjQKJ..."
}
```

FOLDER

```
{
  "operationId": "op-123",
  "name": "Expediente 2026",
  "documentSourceType": "FOLDER"
}
```

EXTERNAL_URL

```
{
  "operationId": "op-123",
  "name": "Referencia externa",
  "documentSourceType": "EXTERNAL_URL",
  "url": "https://example.com/documento/123"
}
```

Upstream calls

1. For DIGITAL requests:
 - POST /uploads
 - PUT {upload_location}
 - POST /files/{process_id}/documents-and-folders/{folder_id}
2. For FOLDER requests:
 - POST /files/{process_id}/documents-and-folders/{folder_id}
3. For EXTERNAL_URL requests:
 - POST /files/{process_id}/documents-and-folders/{folder_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: UploadDocumentResult

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: UploadDocumentError

Notes

- This route uses the same document creation flows as the file-level route, but targets the upstream Gestion endpoint with the folder id in the last path segment.
- For DIGITAL uploads, either fileName or content must be provided.
- If both fileName and content are provided, the current implementation uses content.
- On successful create operations, result.id may come either from the upstream id field or, when that field is missing, from the last segment of the upstream self link.

6. GET /documents/{document_id}

Downloads a document from Gestion. This is an absolute route and is not prefixed by /processes.

Route parameters

- `document_id` required

Query parameters

- `operationId` optional

Request body model

- none

Upstream calls

1. GET `/content/small/documentinstances/{document_id}`

Success response

- HTTP 200 OK
- Body: raw binary document content
- Headers:
 - Content-Type: document MIME type returned by Gestiona, or `application/octet-stream` as fallback
 - Content-Disposition: `attachment; filename=...`
 - X-Operation-Id: present when `operationId` is provided in the request
 - X-Storage-Extension: present when the upstream document metadata includes a storage extension

Download filename resolution

The controller chooses the download filename in this order:

1. `document.fileName`
2. `document.documentId` + "." + `document.storageExtension`
3. `document.documentId`

Validation behavior

- If `document_id` is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as `{{document_id}}`, the endpoint returns HTTP 400

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `UploadDocumentError`

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 404,
    "name": "Not Found",
    "kind": "NotFound",
    "message": "Failed to download document from Gestiona: 12345."
  }
}
```

Notes

- Successful downloads do not return JSON
- Download errors reuse the same `GatewayResponse` envelope as upload errors
- When `operationId` is provided in the download request, it is echoed back:
 - in the error JSON body on failure
 - in the X-Operation-Id response header on success

- When the upstream response includes document storage extension metadata, it is exposed in the X-Storage-Extension response header

7. GET /thirds?nif=<nif>

Gets a third from Gestiona by resolving the third id from a NIF, then enriches it with the default address.

Route parameters

- none

Query parameters

- nif required
- operationId optional

Request body model

- none

Upstream calls

1. GET /thirds with body {"nif": "nif"} and Content-Type: application/vnd.gestiona.filter.thirds+json
2. GET /thirds/{resolved_third_id}
3. GET /thirds/{resolved_third_id}/default-address

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ThirdResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "full_name": "Luis Silva Fernandes",
    "first_name": "Luis",
    "second_surname": "Fernandes",
    "nif_country": "ESP",
    "id": "4b18954c-b66c-4e55-af6d-acf6a2c7aaa3",
    "nif": "196510880",
    "type": "PHISIC",
    "email": "luis.mcf.silva@gmail.com",
    "mobile": "913347827",
    "nif_type": "OWN",
    "address": "Rua das Cancelas",
    "number": "184",
    "zip_code": "4440368",
    "province": "PORTO",
    "country": "Portugal",
    "type_of_road": "CL",
    "zone": "string | null",
    "parish_code": "string | null"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ThirdError

Notes

- The upstream NIF filter response must contain exactly one item in content
- The third id is extracted from the `id` field of that single item
- If `nif` is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as `{{nif}}`, the endpoint returns HTTP 400

8. GET /thirds/{third_id}

Gets a third from Gestiona and enriches it with the default address.

Route parameters

- `third_id` required

Query parameters

- `operationId` optional

Request body model

- none

Upstream calls

1. GET /thirds/{third_id}
2. GET /thirds/{third_id}/default-address

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ThirdResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "full_name": "Leonor Ranito Silva",
    "first_name": "Leonor",
    "second_surname": "Silva",
    "nif_country": "PT",
    "id": "3aeff9c7-a865-4f1a-9cd6-47993b423873",
    "nif": "211211211",
    "type": "PHISIC",
    "email": "leonor.silva@gmail.com",
    "mobile": "913344671",
    "nif_type": "OWN",
    "address": "Rua das Cancelas",
    "number": "184",
    "zip_code": "4440368",
    "province": "PORTO",
    "country": "Portugal",
    "type_of_road": "CL",
    "zone": "string | null",
    "parish_code": "string | null"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ThirdError

Notes

- If `third_id` is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as `{{third_id}}`, the endpoint returns HTTP 400

9. GET /processes?process_number=<numero>

Resolves the Gestion file id associated with `process_number`.

Route parameters

- none

Query parameters

- `process_number` required
- `operationId` optional

Request body model

- none

Upstream calls

1. GET /files with filter `exact_code = process_number`

Success response

- HTTP 200 OK
- Body model: `GatewayResponse`
- result shape: `ProcessResult`

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "Id": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "processNumber": "PROC-2026-001"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: `GatewayResponse`
- result shape: `ProcessError`

Notes

- Id is the resolved Gestion file id, not the original `process_number`
- If no Gestion file is found for `process_number`, the endpoint returns HTTP 404
- If `process_number` is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as `{{process_number}}`, the endpoint returns HTTP 400

10. GET /processes/thirds?process_number=<numero>

Gets the third identifiers associated with a Gestion process file resolved from `process_number`.

Route parameters

- none

Query parameters

- process_number required
- operationId optional

Request body model

- none

Upstream calls

1. GET /files with filter exact_code = process_number
2. GET /files/{resolved_process_id}/thirdparties

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ProcessThirdsResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "processId": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "thirds": "3aeff9c7-a865-4f1a-9cd6-47993b423873;4b18954c-b66c-4e55-af6d-acf6a2c7aaa3;ece5762f-ae00-4da4-a869-a"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessThirdsError

Notes

- The returned processId is the resolved Gestion file id, not the original process_number
- If no Gestion file is found for process_number, the endpoint returns HTTP 404
- If process_number is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{process_number}}, the endpoint returns HTTP 400

11. GET /processes/{process_id}/thirds

Gets the third identifiers associated with a Gestion process file.

Route parameters

- process_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/thirdparties

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: ProcessThirdsResult

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": {
    "processId": "30bcb012-47e2-4e7e-92e0-a0f7278b52b8",
    "thirds": "3aeff9c7-a865-4f1a-9cd6-47993b423873;4b18954c-b66c-4e55-af6d-acf6a2c7aaa3;ece5762f-ae00-4da4-a869-a"
  }
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessThirdsError

Notes

- The service reads the upstream content array and, for each item, finds the link where rel is third
- The third id is extracted from the last segment of that link's href
- The returned thirds field joins all extracted third ids with semicolons
- If process_id is empty or whitespace, the endpoint returns HTTP 400
- If Postman sends an unresolved variable such as {{process_id}}, the endpoint returns HTTP 400

12. GET /processes/{process_id}/documents

Gets the documents and folders at the root of a Gestion process file.

Route parameters

- process_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/documents-and-folders

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ProcessDocumentItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "type": "DOC",
      "name": "POC_SIGMA_Gestiona",
      "id": "f5e0364b-9951-449d-8d38-34a6cbfec4d3"
    },
    {
      "type": "FOLDER",
      "name": "xxxx",
      "id": "12fb9b74-2111-417f-ae4f-c2b7fe2976f7"
    }
  ]
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessDocumentsError

Notes

- Both DOC and FOLDER entries from the upstream content array are returned.
- Each upstream rel is mapped to name.
- Each id is extracted from the last path segment of the upstream href.
- If process_id is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved variable such as {{process_id}}, the endpoint returns HTTP 400.

13. GET /processes/{process_id}/documents/{document_id}

Gets the documents and folders contained inside the specified Gestiona document or folder.

Route parameters

- process_id required
- document_id required

Query parameters

- operationId optional

Request body model

- none

Upstream calls

1. GET /files/{process_id}/documents-and-folders/{document_id}

Success response

- HTTP 200 OK
- Body model: GatewayResponse
- result shape: array of ProcessDocumentItem

Success example

```
{
  "operationId": "op-01",
  "success": true,
  "result": [
    {
      "type": "DOC",
      "name": "Nested document",
      "id": "347d4226-093f-412d-8376-e36d36374d13"
    }
  ]
}
```

Error response

- HTTP 400, 404, 500, or propagated upstream status code
- Body model: GatewayResponse
- result shape: ProcessDocumentsError

Notes

- The response mapping is identical to GET /processes/{process_id}/documents.
- Both DOC and FOLDER entries from the upstream content array are returned.
- If either route parameter is empty or whitespace, the endpoint returns HTTP 400.
- If Postman sends an unresolved {{process_id}} or {{document_id}} variable, the endpoint returns HTTP 400.